Assignment: Data Contracts & YAML

Course: Data Engineering Systems

Topic: Data Quality, Governance, and Contract Authoring

Estimated Time: 120–150 Minutes

Assignment Overview

In this assignment, you will write **four (4)** distinct data contracts in YAML format.

> **Scenario 1** is a comprehensive case study requiring deep attention to logical mapping, negotiation documentation, and quality rules.
>
> **Scenarios 2, 3, and 4** are rapid-fire exercises focusing on specific industry challenges (E-commerce, IoT, FinTech).

⚠️ **Important:** Validate your YAML syntax using an online validator (e.g., yamllint.com) before submission. Invalid YAML structure will result in point deductions.

Scenario 1: The "Ride-Share" Data Contract (Comprehensive)

Context

You are a Data Engineer at **CityMove**, a ride-sharing platform.

Currently, the **"Dynamic Pricing Algorithm" (Machine Learning Team)** consumes data directly from your operational PostgreSQL database. Last week, your team made a routine schema change (renaming a column from `cost_total` to `fare_final`), which caused the pricing algorithm to crash for 4 hours, costing the company $50,000.

To prevent this from happening again, the CTO has mandated the implementation of a **Data Contract** that acts as a stable interface between the database and all consumers.

1.1 The Input Data (Physical Layer — Current State)

Your raw database table (`raw_rides_log_v2`) currently looks like this. Note that the names are cryptic, and there are no constraints. The table below reflects the *current* schema after the recent rename:

db_col_name	Data Type	Sample Value	Notes
r_id	VARCHAR	"a1-b2-c3"	Unique Ride ID
ts_start	TIMESTAMP	2023-10-27 08:30:00	Pickup time (UTC)
pax_id	INT	99821	Passenger ID (Contains PII)
d_rating	FLOAT	4.8	Driver Rating (1.0 – 5.0)
fare_final	FLOAT	24.50	Total fare in USD (was cost_total)
dist_m	INT	5200	Distance traveled in meters

1.2 The Requirements (From Negotiation)

You sat down with the ML Team (Consumer), and they provided the following requirements for the contract:

1. Logical Naming: They want readable, business-friendly names in the contract. For example, instead of `ts_start`, use `pickup_timestamp`.

2. Data Quality (Critical):

- `fare_final` (Fare) must **never** be negative.
- `d_rating` (Driver Rating) must be between 1.0 and 5.0 (inclusive).
- `dist_m` (Distance) must not be null.

3. Privacy: The `pax_id` allows identification of users. This must be flagged as PII (Personally Identifiable Information).

4. SLA (Freshness): The model retrains every hour. They need data to be available within **30 minutes** of a ride completing.

1.3 Your Task

Create a file named **`rides_contract.yaml`** that acts as the interface between your database and the ML team.

Your contract must include:

- **Logical Mapping:** Rename all cryptic DB columns to business-friendly names.
- **Quality Rules:** Implement validation rules for fare, rating, and distance.
- **PII Tagging:** Mark the passenger ID field appropriately.
- **SLA Definition:** Specify the freshness requirement.
- **Negotiation Summary:** Add a brief comment block at the top documenting the producer (your team) and consumer (ML team), plus any trade-offs agreed upon.

> 💡 **Circuit Breaker Hint:** If `fare_final` comes in as -5.00, your contract is the only thing stopping that bad data from breaking the ML model. Think about what enforcement action should occur.

Scenario 2: The "Flash Sale" Order Stream (E-commerce)

Context

It is Black Friday. The Marketing Dashboard Team needs a real-time feed of orders to adjust ad spend dynamically.

The Physical Data

Table `orders_raw` has columns:

- `o_total` (float) — Order total in USD
- `status_code` (int) — 2=Paid, 5=Shipped, 9=Cancelled

The Issue

Last year, a bug caused negative order totals. Also, the dashboard crashed when it received `status_code: 7` because it only understood "Paid" or "Cancelled" text logic — it had no handler for unmapped codes.

Your Task

Create a file named **`orders_contract.yaml`**.

1. **Quality Rule:** Ensure `o_total` is always >= 0.
2. **Schema Enum:** Map the physical `status_code` to a logical string enum field `status` with allowed values: [PAID, SHIPPED, CANCELLED].
3. **Invalid Code Handling:** Add a quality rule that **rejects** any record with an unmapped status_code (i.e., codes other than 2, 5, or 9 should fail validation).

Scenario 3: The "Smart Thermostat" Fleet (IoT)

Context

A fleet of 10,000 smart thermostats sends temperature data every minute to a central analytics platform.

The Physical Data

JSON payload: `{ "dev_id": "xyz", "temp_c": 9999, "batt_lvl": 0.1 }`

The Issue

When sensors fail or lose calibration, they default to sending 9999°C. This breaks the "Average Home Temperature" report by skewing the mean. Additionally, battery level readings outside the valid range (e.g., 1.5 or -0.2) indicate sensor malfunction.

Your Task

Create a file named **`thermostat_contract.yaml`**.

1. **Temperature Range Check:** Write a quality rule that rejects temperature readings outside the range of **-30°C to 60°C (inclusive)**. That is: -30 ≤ temp_c ≤ 60.
2. **Battery Level Check:** Ensure battery level is between **0.0 and 1.0 (inclusive)**. That is: 0.0 ≤ batt_lvl ≤ 1.0. A reading of exactly 0.0 (dead battery) or 1.0 (full charge) is valid.

Scenario 4: The "FinTech" Transaction Log (Financial)

Context

A bank's fraud detection system scans transactions in real-time to identify suspicious activity.

The Physical Data

Table `tx_log` has columns:

- `acct_src` (string) — Source account ID
- `acct_dest` (string) — Destination account ID
- `tx_ts` (timestamp) — Transaction timestamp

The Issue

Due to a legacy system upgrade, some `acct_src` values are coming in as 8 characters instead of the required 10 characters. This causes the fraud model to silently fail lookups, missing potentially fraudulent transactions.

Account ID Format Specification

Valid account IDs must be **exactly 10 characters**, consisting of **uppercase letters (A-Z) and digits (0-9) only**.

✓ Valid: `ABCD123456`, `1234567890`, `A1B2C3D4E5`

✗ Invalid: `abcd1234` (lowercase, too short), `ABC_123456` (contains underscore), `ABC123` (too short)

Your Task

Create a file named **`fintech_contract.yaml`**.

1. Pattern Matching: Write a contract rule using regex to ensure `account_source_id` matches the pattern: exactly 10 uppercase alphanumeric characters.

2. Hard Circuit Breaker: Add an `enforcement` field or comment explicitly indicating this rule requires a "Hard Circuit Breaker" — i.e., any violation must **block the pipeline** rather than just logging a warning.

Submission & Grading

Submission Format

Please submit **four (4)** separate YAML files:

1. `rides_contract.yaml`
2. `orders_contract.yaml`
3. `thermostat_contract.yaml`
4. `fintech_contract.yaml`

Grading Rubric (Total: 100 Points)

Section	Points	Criteria
Scenario 1: Ride-Share	40	• Logical renaming of all columns (10 pts)• Correct implementation of 3 quality rules (12 pts)• PII tagging and SLA definition (10 pts)• Negotiation summary comment block (8 pts)
Scenario 2: E-commerce	20	• o_total non-negative check (7 pts)• Correct enum implementation for status codes (7 pts)• Invalid status_code rejection rule (6 pts)
Scenario 3: IoT	20	• Temperature range validation (-30 to 60 inclusive) (10 pts)• Battery level percentage check (0.0 to 1.0 inclusive) (10 pts)
Scenario 4: FinTech	20	• Regex/pattern check for 10-char uppercase alphanumeric (15 pts)• Hard circuit breaker annotation (5 pts)

Appendix A: Data Contract Template

Use this reference template to structure your assignment. Pay careful attention to the YAML indentation — incorrect indentation will cause parsing errors.

```yaml
#
================================================================================
# DATA CONTRACT TEMPLATE
# Based on Open Data Contract Standard (ODCS) pattern
#
================================================================================

dataContractSpecification: 0.9.3
id: urn:datacontract:[DOMAIN]:[DATASET_NAME]
version: 1.0.0

#
--------------------------------------------------------------------------------
# 1. OWNERSHIP & GOVERNANCE
#
--------------------------------------------------------------------------------
info:
  title: "[Human Readable Dataset Name]"
  description: |
    [High-level summary of what this dataset represents.
    Include business context and primary use cases.]
  owner: "[Team Name]"
  contact:
    name: "[Point of Contact]"
    email: "[team-email@company.com]"
    slack: "#[channel-name]"
  classification: [public | internal | confidential | restricted]
  tags: [tag1, tag2, key-business-entity]

#
--------------------------------------------------------------------------------
# 2. SCHEMA DEFINITION (Logical Contract)
# Define the LOGICAL interface, not the PHYSICAL storage.
# Use business-friendly names, not database abbreviations.
#
--------------------------------------------------------------------------------
schema:
  type: object
  properties:

    # Example: Standard Identifier
    field_name_logical:
      type: string
      description: "What is this? Unique ID? Foreign Key?"
      pii: false
      # physical_mapping: source_column_name  # Optional: for debugging

    # Example: Numeric Metric with Constraints
    field_name_metric:
      type: number
      description: "Business definition, e.g., Gross revenue in USD"
      minimum: 0

    # Example: Categorical Data (Enum)
```

```
    field_name_status:
      type: string
      enum: [value_1, value_2, value_3]
      description: "Allowed state values"

  required: [list, of, mandatory, fields]

#
------------------------------------------------------------------------------
----------
# 3. SERVICE LEVEL AGREEMENTS (SLAs)
#
------------------------------------------------------------------------------
----------
sla:
  freshness:
    threshold: "15 minutes"
    description: "Time from source event to data availability"
  availability:
    percentage: "99.9%"
    description: "Uptime commitment"
  retention:
    period: "2 years"
    description: "How long is history preserved?"

#
------------------------------------------------------------------------------
----------
# 4. DATA QUALITY RULES
# Executable rules to validate data expectations.
#
------------------------------------------------------------------------------
----------
quality:
  - name: rule_name_uniqueness
    type: uniqueness
    column: primary_key_field
    threshold: "100%"

  - name: rule_name_completeness
    type: completeness
    column: important_field
    threshold: "99.5%"

  - name: rule_name_validity
    type: validity
    expression: "amount >= 0"
    threshold: "100%"

  - name: rule_name_range_check
    type: validity
    expression: "value >= -30 AND value <= 60"
    threshold: "100%"

  - name: rule_name_pattern
    type: validity
    expression: "REGEXP_LIKE(field, '^[A-Z0-9]{10}$')"
    threshold: "100%"
    enforcement: hard  # Options: hard (block pipeline), soft (warn only)
```

Appendix B: Pre-Flight Checklist

Before you finalize your YAML files, ensure you can answer "Yes" to these questions.

Phase 1: Negotiation & Scope (The "Who" & "What")

☐ **Identify the Parties:** Have you clearly defined who is the Producer and who is the Consumer?

☐ **Document the Negotiation:** Did you include a comment block summarizing the agreement and any trade-offs?

☐ **Realistic Thresholds:** Are the quality thresholds achievable by the producer?

Phase 2: Contract Definition (The "How")

☐ **Logical vs. Physical:** Are field names user-friendly (Logical), not cryptic DB names (Physical)?

☐ **Semantic Clarity:** Does every field have a description explaining its business meaning?

☐ **PII & Governance:** Are PII fields explicitly tagged?

☐ **YAML Valid:** Did you run your file through a YAML validator?

Phase 3: Enforcement Strategy (The "Circuit Breaker")

☐ **Validation Points:** Where will you enforce this contract? (Ingestion? Publication? Consumer read?)

☐ **Failure Policy:** What happens when a rule breaks? (Hard = block pipeline, Soft = warn only)

☐ **Edge Cases:** Have you considered boundary conditions (e.g., exactly 0.0 battery, exactly 10-char IDs)?

Appendix C: Worked Example — HR Payroll Contract

This appendix provides a **complete worked example** of a data contract for a scenario similar in complexity to Scenario 1. Use this as a reference for structure, style, and level of detail expected.

> **Note:** This example uses HR/Payroll data — a domain deliberately different from all four assignment scenarios (transportation, e-commerce, IoT, financial transactions).

Example Scenario: "TechCorp" Employee Payroll System

Context

You are a Data Engineer at **TechCorp**, a mid-size technology company. The **"Workforce Analytics Team" (HR Department)** needs reliable employee compensation data to generate headcount reports, salary benchmarking analysis, and compliance audits.

Last quarter, a payroll system migration caused some records to have invalid department codes and corrupted salary figures (negative values). This broke the quarterly compensation report and triggered a compliance investigation.

Physical Schema

Table `emp_payroll_raw` has the following columns:

db_col_name	Data Type	Sample Value	Notes
e_id	VARCHAR	"EMP-00742"	Employee ID (format: EMP-NNNNN)
ssn_enc	VARCHAR	"x9a8b7c6..."	Encrypted SSN (highly sensitive PII)
dept_cd	VARCHAR	"ENG"	Department: ENG, MKT, SAL, HR, FIN, OPS
sal_annual	FLOAT	95000.00	Annual salary in USD
dt_hire	DATE	2021-03-15	Hire date
dt_term	DATE	NULL	Termination date (NULL if active)
emp_lvl	INT	3	Level: 1=Junior, 2=Mid, 3=Senior, 4=Lead, 5=Director
fte_pct	FLOAT	1.0	Full-time equivalent (0.0–1.0)

Negotiated Requirements

After discussion between the Data Engineering team (Producer) and the HR Analytics team (Consumer):

- Logical, readable field names required (no abbreviations)
- Employee ID must match pattern EMP-NNNNN (EMP- followed by exactly 5 digits)
- Annual salary must be between $30,000 and $500,000 (company policy bounds)
- Department codes must be valid enum; unmapped codes rejected
- FTE percentage must be 0.0–1.0 inclusive
- Termination date (if present) must be after hire date
- SSN is highly sensitive PII — must be tagged and access-restricted
- Data freshness: 24 hours (payroll runs nightly)

Example Solution: payroll_contract.yaml

Below is the complete, valid YAML contract that satisfies all requirements:

```
#
═══════════════════════════════════════════════════════════════════════════════
═══════
# DATA CONTRACT: TechCorp Employee Payroll
#
═══════════════════════════════════════════════════════════════════════════════
═══════
#
# NEGOTIATION SUMMARY
# ──────────────────────────
# Producer: Data Engineering Team (data-platform@techcorp.com)
# Consumer: HR Workforce Analytics (hr-analytics@techcorp.com)
#
# Agreed Terms:
#   - Producer commits to 24-hour freshness (aligned with nightly payroll batch)
#   - Salary bounds ($30K-$500K) agreed as company policy; outliers rejected
#   - Hard enforcement on all rules due to compliance requirements
#   - SSN field included but marked restricted; consumers must have HR clearance
#   - Employee level mapped from integers to readable titles at contract layer
#
# Compliance Note: This data subject to SOX audit requirements
# Signed off: 2024-02-01 by Data Eng Lead + HR Director
#
═══════════════════════════════════════════════════════════════════════════════
═══════


dataContractSpecification: 0.9.3
id: urn:datacontract:techcorp:employee-payroll
```

```yaml
version: 1.0.0

# ------------------------------------------------------------------------------
# 1. OWNERSHIP & GOVERNANCE
# ------------------------------------------------------------------------------
info:
  title: "TechCorp Employee Payroll"
  description: |
    Employee compensation and employment status records.
    Includes salary, department, level, and employment dates.
    Primary use: Headcount reporting, salary benchmarking, compliance audits.
    WARNING: Contains highly sensitive PII (SSN). Access restricted to HR.
  owner: "Data Engineering Team"
  contact:
    name: "HR Data Steward"
    email: "hr-data@techcorp.com"
    slack: "#hr-data-support"
  classification: restricted  # Highest sensitivity due to SSN and salary
  tags: [hr, payroll, compensation, pii, compliance, sox]

# ------------------------------------------------------------------------------
# 2. SCHEMA DEFINITION (Logical Contract)
# ------------------------------------------------------------------------------
schema:
  type: object
  properties:

    employee_id:
      type: string
      pattern: "^EMP-[0-9]{5}$"
      description: |
        Unique employee identifier.
        Format: 'EMP-' followed by exactly 5 digits (e.g., EMP-00742).
      pii: false
      # physical_mapping: e_id

    social_security_number:
      type: string
      description: |
        Encrypted Social Security Number. HIGHLY SENSITIVE.
        Access restricted to HR personnel with compliance clearance.
        Do not log, cache, or display in dashboards.
      pii: true  # RESTRICTED: Requires HR clearance to access
      # physical_mapping: ssn_enc

    department:
      type: string
      enum: [ENGINEERING, MARKETING, SALES, HUMAN_RESOURCES, FINANCE, OPERATIONS]
      description: |
        Employee's department.
        Mapped from codes: ENG=ENGINEERING, MKT=MARKETING, SAL=SALES,
```

```
      HR=HUMAN_RESOURCES, FIN=FINANCE, OPS=OPERATIONS
    pii: false
    # physical_mapping: dept_cd (3-letter codes)

  annual_salary_usd:
    type: number
    minimum: 30000
    maximum: 500000
    description: |
      Annual base salary in USD. Does not include bonuses or equity.
      Valid range: $30,000 - $500,000 per company compensation policy.
    pii: true  # Salary is considered sensitive PII
    # physical_mapping: sal_annual

  hire_date:
    type: string
    format: date
    description: "Date employee started at the company (YYYY-MM-DD)"
    pii: false
    # physical_mapping: dt_hire

  termination_date:
    type: string
    format: date
    description: |
      Date employment ended (YYYY-MM-DD).
      NULL if employee is currently active.
    pii: false
    # physical_mapping: dt_term

  employee_level:
    type: string
    enum: [JUNIOR, MID, SENIOR, LEAD, DIRECTOR]
    description: |
      Career level/seniority.
      Mapped from integers: 1=JUNIOR, 2=MID, 3=SENIOR, 4=LEAD, 5=DIRECTOR
    pii: false
    # physical_mapping: emp_lvl (integer 1-5)

  fte_percentage:
    type: number
    minimum: 0.0
    maximum: 1.0
    description: |
      Full-Time Equivalent as decimal.
      1.0 = full-time, 0.5 = half-time, etc.
      Valid range: 0.0 to 1.0 inclusive.
    pii: false
    # physical_mapping: fte_pct

required:
  - employee_id
  - department
  - annual_salary_usd
  - hire_date
  - employee_level
  - fte_percentage
```

```
# ──────────────────────────────────────────────────────────────────────────────
# 3. SERVICE LEVEL AGREEMENTS (SLAs)
# ──────────────────────────────────────────────────────────────────────────────
sla:
  freshness:
    threshold: "24 hours"
    description: "Data refreshed nightly after payroll batch completes"
  availability:
    percentage: "99.9%"
    description: "High availability required for compliance reporting"
  retention:
    period: "7 years"
    description: "Retained per SOX and labor law requirements"

# ──────────────────────────────────────────────────────────────────────────────
# 4. DATA QUALITY RULES
# ──────────────────────────────────────────────────────────────────────────────
quality:

  # Rule 1: Employee ID format validation
  - name: employee_id_format
    type: validity
    expression: "REGEXP_LIKE(employee_id, '^EMP-[0-9]{5}$')"
    threshold: "100%"
    enforcement: hard
    description: "Employee ID must match pattern EMP-NNNNN"

  # Rule 2: Employee ID uniqueness
  - name: employee_id_unique
    type: uniqueness
    column: employee_id
    threshold: "100%"
    enforcement: hard
    description: "Each employee must have a unique identifier"

  # Rule 3: Salary within policy bounds
  - name: salary_within_bounds
    type: validity
    expression: "annual_salary_usd >= 30000 AND annual_salary_usd <= 500000"
    threshold: "100%"
    enforcement: hard  # Compliance critical
    description: "Salary must be within company policy range ($30K-$500K)"

  # Rule 4: FTE percentage valid range
  - name: fte_percentage_valid
    type: validity
    expression: "fte_percentage >= 0.0 AND fte_percentage <= 1.0"
    threshold: "100%"
    enforcement: hard
    description: "FTE must be between 0.0 and 1.0 inclusive"
```

```yaml
  # Rule 5: Department must be valid enum
  - name: department_valid_enum
    type: validity
    expression: |
      department IN ('ENGINEERING', 'MARKETING', 'SALES',
                     'HUMAN_RESOURCES', 'FINANCE', 'OPERATIONS')
    threshold: "100%"
    enforcement: hard
    description: "Only valid department values accepted; unmapped codes rejected"

  # Rule 6: Employee level must be valid enum
  - name: employee_level_valid_enum
    type: validity
    expression: "employee_level IN ('JUNIOR', 'MID', 'SENIOR', 'LEAD', 'DIRECTOR')"
    threshold: "100%"
    enforcement: hard
    description: "Only valid level values accepted"

  # Rule 7: Termination date must be after hire date (if present)
  - name: termination_after_hire
    type: validity
    expression: "termination_date IS NULL OR termination_date > hire_date"
    threshold: "100%"
    enforcement: hard
    description: "Cannot terminate before hire date; logical consistency check"

  # Rule 8: Hire date not in future
  - name: hire_date_not_future
    type: validity
    expression: "hire_date <= CURRENT_DATE"
    threshold: "100%"
    enforcement: hard
    description: "Hire date cannot be in the future"

#
──────────────────────────────────────────────────────────────────────────────────────────────
──────────
# 5. LINEAGE & CONSUMERS
#
──────────────────────────────────────────────────────────────────────────────────────────────
──────────
lineage:
  sources:
    - name: hris_payroll_db
      type: postgresql
      table: emp_payroll_raw
      description: "HR Information System payroll database"
  consumers:
    - name: headcount-dashboard
      team: hr-analytics
      usage: "Monthly headcount and department distribution reports"
    - name: compensation-benchmarking
      team: hr-analytics
      usage: "Quarterly salary analysis and market comparison"
    - name: sox-compliance-audit
      team: internal-audit
      usage: "Annual SOX compliance reporting"
```

Key Takeaways from This Example

1. Negotiation Summary: The comment block documents parties, agreed terms, compliance notes, and sign-off — including who approved the contract.

2. Logical Naming: All cryptic names (e_id, sal_annual, dept_cd) are mapped to readable names (employee_id, annual_salary_usd, department).

3. Pattern Validation: The employee_id field uses regex (^EMP-[0-9]{5}$) to enforce the exact format — similar to what you need in Scenario 4.

4. Range Validation: Salary uses both minimum and maximum bounds; FTE uses 0.0–1.0 inclusive — similar to Scenario 3's requirements.

5. Multiple PII Fields: Both SSN (highly sensitive) and salary (sensitive) are tagged as pii: true with appropriate warnings.

6. Enum Mapping: Both department codes (ENG→ENGINEERING) and level integers (1→JUNIOR) are mapped to readable strings.

7. Temporal Logic: The termination_after_hire rule shows how to validate date relationships with NULL handling.

8. All Hard Enforcement: Due to compliance requirements (SOX), all rules use enforcement: hard — appropriate for regulated data.